Nicholas Sima, Ilay Guler, Jimmy Hou

Professor Czik

CS501 Mobile App Development

5/5/2026

Lumenate Final Report

Lumenate is an assisted sight application that allows users to navigate around a closed environment. Once the app has been set up and the user is ready to begin, it will alert them of nearby objects and even give emergency alerts for objects within 0.2 meters or less. Every screen and function of Lumenate is fully accessible with voice commands, making it perfect for any degree of visual disabilities.

Architecture

The code follows an easy-to-understand structure which details exactly what each module does. We used a NavController to select which composables must be on the screen during execution, and created 5 main composable functions: Onboarding, Blurb, Camera, Settings, and Help. Onboarding allows our user a chance to understand the app's purpose and grant the necessary permissions to access the device's camera and microphone. Blurb gives the user the app's entire functionality including exactly how to use it. Camera is where the main functionality comes in: Lumenate uses multiple helpers from Google including their Cloud Text-to-Speech service and an image analysis service, and also incorporates some of Android's built-in libraries alongside a third party machine learning model.

Feature Completeness including API & Sensors

The object detections features for this application are implemented using the TensorFlow Lite (TFLite) library, alongside EfficientDet, an ML model trained on the CoCo dataset capable of classifying around 90 objects. The depth calculations were implemented using Google's ARCore Depth API, which creates a depth map, with a depth value and confidence value for each pixel's depth from the user. For each object detected, we took the labeled pixels of that object, based on the coordinates of its bounding box, and then the median depth from all pixels within that object to get a distance measurement of how far the object is from the user. When the closest object is less than a certain distance away (around 0.2 meters), a proximity warning is issued, and the phone uses its vibration motor to vibrate the phone for 1 second, and Google's TTS speaks to the user that the object is less than that certain distance away. The camera itself is also using ARCore's Camera API, instead of CameraX, which was not compatible with ARCore. Lastly, the object's relative bearings were calculated by measuring where the bounding box of the object was located in reference to the center of the image. Through using a phone's built-in camera, alongside the device's inertial measurement unit (accelerometer, gyroscope, and magnetometer), we can detect objects and measure their distance in a 3D space. Through using the device's speakers and vibration motor, we can communicate object distances and warnings to the user. Lastly, through using the device's microphone, the user has an accessible way to interact with the application. We also added a Settings Menu so that users can adjust how many objects they can detect and how fast they want it to be detected.

Text-to-speech was handled by Google's Cloud Text-to-Speech API and makes use of 3 of their voices. The user is able to choose among any of these 3 voices for all necessary communication. Additionally, Android's built in SpeechRecognizer with continuousSpeechFlow was used for speech-to-text capabilities and allows the user to navigate any screen with just their voice while

ensuring a seamless flow of experience. We had previously used Android's TTS engine for text-to-speech, but after switching to an API we had to implement a MediaPlayer to handle the playing of Google's transmitted mp3s.

Team Responsibilities and Contributions

Ilay implemented object detection using TFLite, originally with cameraX, which was corrected by Jimmy to make use of ARCore; this was done by replacing the input of the object detection model from cameraX's specific Image object to a generic Image object. He also manually processed the image so that the image can be easily fed into the model for detection. Next, after Jimmy's first iteration of DepthAPI, Ilay improved on this by adding a function which selects from raw depth api, or smooth depth api as a fallback if raw depth is not available. Through utilizing raw depth, we were able to get far more accurate depth measurements than before. Lastly, Ilay added the helper methods to allow the app to calculate relative bearings for objects based on the position of their bounding box in relation to the center of the image.

Jimmy mainly worked on Google's ARCore functionality: Replacing the camera from cameraX to ARCore's camera, adjusting the detection model to work with ARCore, and implementing the Depth/Raw Depth API to the app. He also adjusted the bounding box so that the Depth API can figure out what to measure for the distance of the object. Lastly, Jimmy implemented the Settings Screen to adjust the number of detected objects, and how often they should be detected, and added that state to the User Preferences Repository. He also implemented proximity detection/warnings so that when a user is less than 0.2m away from the closest object, a popup will show, and the phone vibrates, telling the user that they are too close to the object that is x meters away.

Nicholas worked on code architecture, including navigation and lifecycle maintenance, and implemented both the text-to-speech and speech-to-text services including screens to allow any potential somewhat visually capable user to have text artifacts as well. Before the app's main functionality came into place, he provided detailed scaffolding and skeleton code for others to include their function callbacks once implemented. Once the rest of the app came into place (image detection and analysis), he ensured that the lifecycles of each composable and all coroutines could work in unison with no breaks. He was also the GitHub repository owner and worked through all merge conflicts and consolidation of branches into one final product.

AI Disclosure

AI was used to help understand external libraries and APIs without the time cost needed for otherwise reading documentation extensively. Our methods for implementing Google's ARCore Depth API, and TFLite library were greatly expedited by asking AI how we can incorporate these libraries given our application's goals; i.e. which functions in these libraries will help us accomplish our goals, what inputs are necessary from our application to these functions, describe the data type this function outputs, etc.